The first thing to do is tell Spring Boot to scan the application context for configuration data beans. We can do so by adding the `@ConfigurationPropertiesScan` annotation to the `CatalogServiceApplication` class in your Catalog Service project (catalog-service).

Listing 4.1 Enabling scanning of configuration data beans

```
package com.polarbookshop.catalogservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties
➥ .ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan                    ❶
public class CatalogServiceApplication {
  public static void main(String[] args) {
    SpringApplication.run(CatalogServiceApplication.class, args);
  }
}
```

❶ Loads configuration data beans in the Spring context

NOTE Instead of making Spring scan the application context, searching for configuration data beans, you can directly specify which ones Spring should consider by using the `@EnableConfigurationProperties` annotation.

Next, you can define a new `com.polarbookshop.catalogservice.config` package, and create a `PolarProperties` class annotated with `@ConfigurationProperties` to mark it as a holder of configuration data. The `@ConfigurationProperties` annotation takes a `prefix` argument, combined with a field name, to produce the final property key. Spring Boot will try to map all properties with that prefix to fields in the class. In this case, there's only one property mapped to the bean: `polar.greeting`. Optionally, you can add a description for each